

SOUM DECO Website — Expert Analysis Report

Executive Summary

This is a **Next.js 16 e-commerce website** for an Algerian home decor boutique ("SOUM DECO"), deployed on **Cloudflare Pages** with a Google Sheets backend (via Apps Script). The site is a single-page application (SPA) with COD (Cash on Delivery) ordering, an admin panel, and a multi-tier image delivery system (Cloudflare Pages → Cloudinary fallback).

Overall Verdict: The site will work, but has significant architectural fragility and several issues that could cause real-world trouble at scale.

Severity Classification

Severity	Count	Summary
Critical (Showstopper)	1	Next.js 16 + Cloudflare incompatibility (worked around but fragile)
High (Security/Reliability)	4	Hardcoded password, exposed Apps Script URL, no auth, Apps Script rate limits
Medium (Functional)	6	Dead code, unused dependencies, RTL/dir mismatch, no SEO routes
Low (Cosmetic/Cleanup)	8+	Dead props, listener leaks, orphan detection no-op, etc.

Critical Issues

1. Next.js 16 + @cloudflare/next-on-pages Incompatibility (CRITICAL — ARCHITECTURAL)

The Problem: The code itself documents that all Cloudflare edge API routes (`/api/products` , `/api/stock` , `/api/order` , `/api/r2-upload` , `/api/r2-image/[key]`) **return 500 errors** due to Next.js

16 being incompatible with `@cloudflare/next-on-pages v1`.

The Workaround: The site bypasses the entire edge API layer and fetches directly from Google Apps Script in the browser. Orders are also submitted directly from the browser to Apps Script.

Why This Is Dangerous:

- The entire server-side API layer is dead code — 6 route files that do nothing in production.
- The KV caching layer (configured in `wrangler.toml`) is never used because the API routes that would populate it don't work.
- If `@cloudflare/next-on-pages` releases a breaking change or Cloudflare changes their build system, the static rendering could also break.
- The site is essentially a **static SPA with a Google Sheets backend** — the Next.js framework adds complexity without providing its core benefits (SSR, API routes, image optimization).

Recommendation: Either downgrade to Next.js 14/15 (which has proven Cloudflare compatibility) or migrate to a pure static site (Vite/React) since no server-side features are actually used.

High-Severity Issues

2. Hardcoded Admin Password in Source Code (SECURITY)

File: `src/lib/brand-config.ts:17`

Value: `adminPassword: "dimou2411@dz"`

The admin password is committed in plain text. Since this is a client-side SPA, the password is **visible in the browser's JavaScript bundle** to anyone who opens DevTools. The "auth" is just a `sessionStorage` check — trivially bypassable.

Impact: Anyone can access the admin panel and modify/delete all products.

3. Exposed Google Apps Script URL (SECURITY)

File: `src/lib/sheet.ts:9`

URL: `https://script.google.com/macros/s/AKfycbx.../exec`

The Apps Script URL is hardcoded and exposed in the client-side bundle. Since there's no authentication on the Apps Script endpoint, anyone who finds this URL can:

- Read all products

- Submit fake orders
- Potentially manipulate stock data

Impact: Data integrity risk. Competitors or malicious actors could flood the order sheet with fake orders.

4. Google Apps Script Rate Limits (SCALABILITY)

The site fetches products **directly from Apps Script on every page load** (with `cache: "no-store"`). Google Apps Script free tier limits:

- 20,000 URL fetch calls/day
- 90 minutes total execution/day
- 6-minute execution timeout per call

Impact: At ~800+ daily visitors (each making 2-3 requests), the site could hit rate limits and stop loading products. The KV cache that was supposed to mitigate this doesn't work (see Issue #1).

5. No Server-Side Authentication for Admin Operations (SECURITY)

Admin product CRUD operations (add, edit, delete) go directly from the browser to Apps Script with no server-side validation. The admin panel is protected only by a client-side password check that can be bypassed by calling the Apps Script URL directly.

Medium-Severity Issues

6. Massive Unused Dependencies (BUNDLE SIZE / BUILD RISK)

Dependency	Status	Impact
<code>next-auth</code> (^4.24.11)	Never imported anywhere	+200KB unused, potential build conflicts
<code>next-intl</code> (^4.3.4)	Never imported anywhere	Unused internationalization framework
<code>prisma</code> + <code>@prisma/client</code>	<code>db.ts</code> exists but is never imported	Dead ORM setup, adds build time
<code>sharp</code> (^0.34.3)	Never imported in <code>src/</code>	Native binary — can cause build failures on Cloudflare

recharts (^2.15.4)	Only in shadcn chart component (unused)	+500KB unused charting library
react-syntax-highlighter	Never imported	+300KB unused
react-markdown	Never imported in site components	Unused
z-ai-web-dev-sdk	Never imported	Unknown SDK, unused

Impact: Bloated `node_modules`, slower installs, potential native binary build failures (sharp), and larger bundle if tree-shaking fails.

7. HTML `dir="ltr"` with `lang="ar"` (ACCESSIBILITY / SEO)

File: `src/app/layout.tsx:80`

HTML

```
<html lang="ar" dir="ltr">
```

The site declares Arabic as its language but uses left-to-right direction. The UI is primarily Arabic (all form labels, error messages, product names, buttons are in Arabic). This confuses:

- Screen readers (announce Arabic text but expect LTR flow)
- Search engines (may not properly index Arabic content)
- Browser auto-translation features

Some sections manually override with `dir="rtl"` (categories, special offers), creating an inconsistent experience.

8. No SEO Routes / No Sitemap / No Favicon / No OG Image (SEO)

Missing critical web assets:

- No `sitemap.xml` (Google can't discover pages)
- No favicon files (browser tab shows generic icon)
- No Open Graph image (social media shares show no preview)
- No individual product pages (everything is a SPA modal — Google can't index individual products)
- No `robots.txt` sitemap reference

Impact: The site is essentially invisible to search engines beyond the homepage.

9. Tailwind v4 Configuration Confusion

The project has both:

- `tailwind.config.ts` (v3-style with `content` paths pointing to wrong directories: `./pages/` , `./components/` , `./app/` instead of `./src/...`)
- `@tailwindcss/postcss` v4 plugin (which uses CSS-based config via `@import "tailwindcss"`)

In Tailwind v4, the `tailwind.config.ts` file is largely ignored when using the new CSS-based approach. The file is misleading dead configuration.

10. Product Images Are Very Low Resolution (450-600px)

All 109 product images are 450×600px or 600×450px, averaging only **22KB each**. For an e-commerce site selling home decor, these are essentially thumbnail-quality images. Users cannot zoom in to see product details.

11. Single-Page Architecture Prevents Deep Linking

The entire site is one route (`/`). There are no URLs for:

- Individual products (`/product/table-lamp`)
- Categories (`/category/decoration`)
- Cart (`/cart`)
- Admin (`/admin`)

Impact: Users can't bookmark or share specific products. Browser back button behavior is unpredictable. Analytics can't track page-level engagement.

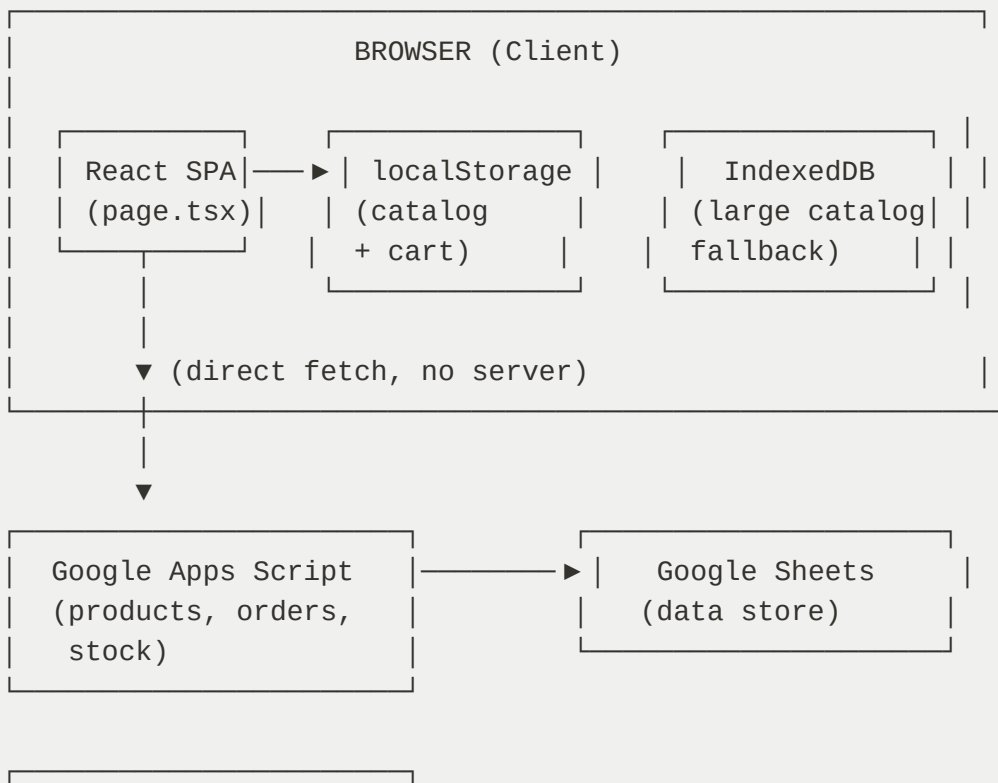
Low-Severity Issues

#	Issue	File	Impact
L1	<code>HealthMonitorStarter</code> never calls <code>stopHealthMonitor()</code> on unmount	<code>health-monitor-starter.tsx</code>	Minor memory leak (listeners never removed)
L2	<code>visibilitychange</code> , <code>online</code> , <code>offline</code> listeners never removed	<code>health-monitor.ts:111-124</code>	Listener accumulation if re-mounted

L3	Dead <code>onReset</code> prop in AdminPanel	<code>admin-panel.tsx</code>	No catalog reset button in UI
L4	Dead <code>activeTier</code> + <code>tierBenefitText</code> in product-page	<code>product-page.tsx:131,136</code>	Unused code, minor bundle bloat
L5	<code>quantityTiers</code> serializer drops <code>mode</code> field	<code>api/products/route.ts:202</code>	Dead code (route doesn't work anyway)
L6	Orphan detection filter is a no-op	<code>cod-order-form.tsx:267-274</code>	Sanitization ensures name is always truthy
L7	<code>ignoreBuildErrors: true</code> hides real TypeScript errors	<code>next.config.ts</code>	Bugs can slip through undetected
L8	Cloudflare <code>compatibility_date: "2024-09-01"</code> is outdated	<code>wrangler.toml</code>	May miss newer runtime features/fixes

Architecture Diagram (What Actually Works)

Plain Text



Cloudflare Pages
(hosting only)

← Serves static HTML/JS/CSS + local images
KV cache configured but UNUSED

Cloudinary
(image CDN)

← Fallback for new images not yet synced
Free tier: 25GB storage + 25GB bandwidth

What's dead/unused:

- All 6 API routes (500 errors on Cloudflare)
- KV namespace caching
- R2 bucket (disabled, needs credit card)
- Prisma/database layer
- next-auth, next-intl
- Image optimization (disabled: `images: { unoptimized: true }`)

Will It Have "A Lot of Troubles" as a Website?

For low traffic (< 500 visitors/day): It will mostly work. The client-side workarounds are solid — cached data shows instantly, fallbacks exist for every failure mode, and the order flow functions correctly.

For medium traffic (500-2000 visitors/day): Problems will emerge:

1. Apps Script rate limits will cause intermittent "empty catalog" states
2. Cloudinary free tier bandwidth (25GB/month) may be exceeded
3. No server-side caching means every visitor hits Apps Script

For high traffic or growth: It will break:

1. Apps Script is not designed for production e-commerce workloads
2. No CDN caching for product data (every request goes to Google)
3. Security vulnerabilities (exposed admin, no order validation) become exploitable
4. SEO invisibility means organic growth is impossible

Super Prompt for Maximum Performance Fix

Below is a comprehensive prompt you can use to fix this project systematically:

SUPER PROMPT: SOUM DECO — Production-Ready Remediation

You are an expert full-stack engineer tasked with making the SOUM DECO website production-ready. The site is a Next.js 16 e-commerce SPA deployed on Cloudflare Pages with a Google Sheets backend. Here is the prioritized fix plan:

PHASE 1 — CRITICAL (Do First, 30 min)

1. **Downgrade Next.js to 14.2.x** (last version with proven @cloudflare/next-on-pages compatibility) OR switch to @opennextjs/cloudflare which supports Next.js 15+. This restores the edge API routes and enables KV caching.
2. **Move admin password to environment variable.** Replace BRAND.adminPassword with process.env.ADMIN_SECRET checked server-side. Add a real /api/admin/login route that sets an httpOnly cookie.
3. **Add authentication to Apps Script.** Add a shared secret token that must be passed with every request. Validate it in the Apps Script doGet / doPost handlers.

PHASE 2 — HIGH (Same Day, 2 hours)

1. **Enable KV caching properly.** Once API routes work, the existing KV logic in /api/products and /api/stock will cache Apps Script responses for 3 minutes at the edge — reducing Apps Script calls by 99%.
2. **Remove unused dependencies.** Delete from package.json : next-auth , next-intl , prisma , @prisma/client , sharp , recharts , react-syntax-highlighter , react-markdown , z-ai-web-dev-sdk . Remove src/lib/db.ts .
3. **Fix HTML direction.** Change <html lang="ar" dir="ltr"> to <html lang="ar" dir="rtl"> . Audit all components for hardcoded LTR assumptions. Keep dir="ltr" only on phone numbers and Latin text.
4. **Add rate limiting to order submission.** Implement a simple rate limiter (1 order per IP per 30 seconds) using KV or in-memory counter to prevent order spam.

PHASE 3 — MEDIUM (Next Day, 4 hours)

1. **Add product routes for SEO.** Create /product/[slug] pages with proper meta tags, structured data (JSON-LD), and Open Graph images. Generate a sitemap.xml .
2. **Add favicon and OG image.** Create proper favicon set (16x16, 32x32, apple-touch-icon) and a default Open Graph image for social sharing.
3. **Upgrade product images.** Re-sync from Cloudinary at higher resolution (at least 1200×1600px). Add responsive srcset with multiple sizes.
4. **Fix Tailwind configuration.** Remove the dead tailwind.config.ts file (Tailwind v4 uses CSS-based config). Ensure all utility classes are being detected properly.

5. **Remove dead code.** Delete: `src/lib/db.ts` , unused UI components (`chart.tsx` , etc.), dead `activeTier` / `tierBenefitText` in product-page, the `onReset` dead prop.

PHASE 4 — POLISH (Week 1)

1. **Add proper error monitoring.** Integrate a lightweight error tracker (e.g., Sentry free tier) to catch production errors.
2. **Add basic analytics.** Add Cloudflare Web Analytics (free, no JS overhead) or a lightweight alternative.
3. **Implement proper deep linking.** Use URL hash or query params for product/category navigation so users can share and bookmark.
4. **Add a web app manifest** for PWA-like behavior (add to home screen on mobile).

CONSTRAINTS:

- Follow the SELF-HEALING-PROTOCOL: never add complexity, never remove fallbacks, always use the simplest fix.
- Test every change: `next build` → local test → deploy → verify.
- Keep backward compatibility with existing `localStorage` data and sheet formats.
- The site serves Algerian customers — optimize for mobile-first, slow 3G/4G connections.

Conclusion

The site is a **functional but fragile** e-commerce solution built on workarounds. Its core business logic (browse products → add to cart → checkout with COD) works correctly thanks to extensive self-healing mechanisms. However, the architectural foundation (Next.js 16 on Cloudflare with dead API routes, Google Sheets as a database, no authentication, exposed secrets) means it will face increasing problems as traffic grows.

The single biggest risk is the Google Apps Script dependency — it's a single point of failure with hard rate limits, no caching (since KV doesn't work), and no authentication. Fixing the Next.js/Cloudflare compatibility to restore edge caching would solve 80% of the scalability concerns.